



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Cano Puerta

TCC-2

BasiliskScan

Presidente Epitácio – SP
2026

SUMÁRIO

Capítulo 1 - DESCRIÇÃO GERAL DO PRODUTO	3
1.1 Escopo do Produto	3
1.2 Funções do Produto	4
1.2.1 Funções Fundamentais	4
1.2.2 Funções Básicas	4
1.2.3 Funções de Saída	4
1.3 Regras do Negócio	5
1.4 Perspectiva do produto / Requisitos não funcionais	5
1.5 Restrições, Suposições e Dependências	5
Capítulo 2 - REQUISITOS ESPECÍFICOS	6
2.1 Diagrama de Casos de Uso	6
2.2 Especificações de Casos de Uso e Diagramas de Atividades	6
Capítulo 3 - PROJETO DE SOFTWARE	22
3.1 Arquitetura Lógica de Software	22
3.2 Diagrama de Classes	23
3.3 Diagramas de Sequência	24
3.4 Mapeamento OO-Relacional (Modelo Lógico de Dados)	30

Capítulo 1 - DESCRIÇÃO GERAL DO PRODUTO

1.1 Escopo do Produto

O sistema BasiliskScan tem como objetivo auxiliar desenvolvedores na identificação de componentes vulneráveis e desatualizados em sistemas web, por meio de uma ferramenta de linha de comando. A aplicação realiza a análise de arquivos de dependência de projetos desenvolvidos em Node.js, Ionic/TypeScript e Java Web, extraíndo nomes e versões das bibliotecas utilizadas.

A partir dessas informações, o BasiliskScan consulta bases públicas de vulnerabilidades, como OSV, NVD e Sonatype Guide, identificando possíveis falhas conhecidas, versões inseguras e recomendações de atualização. Os resultados são apresentados em relatórios HTML, contendo dados como componentes encontrados, vulnerabilidades associadas, identificadores CVE, severidade e sugestões de correção.

O sistema será utilizado principalmente por desenvolvedores e equipes responsáveis pela manutenção de projetos da Fábrica de Software Acadêmica, contribuindo para maior controle das dependências utilizadas e para a redução de riscos de segurança. A ferramenta não realizará correções automáticas no código nem explorará vulnerabilidades, limitando-se à análise, identificação e apresentação de informações que auxiliem a tomada de decisão.

A implantação do BasiliskScan permitirá que os projetos analisados tenham maior visibilidade sobre seus componentes de terceiros, favorecendo práticas de desenvolvimento mais seguras, organizadas e alinhadas à prevenção do uso de bibliotecas vulneráveis ou obsoletas.

1.2 Funções do Produto

1.2.1 Funções Fundamentais

RF_FF01 - Realizar Varredura de Projeto. Itens de informação necessários: Caminho do Projeto, Ecossistema do Projeto, Arquivos de Dependência, Modo de Execução.

RF_FF02 - Identificar Componentes do Projeto. Itens de informação necessários: Nome do Componente, Versão, Tipo de Componente, Arquivo de Origem, Ecossistema.

RF_FF03 - Consultar Vulnerabilidades Conhecidas. Itens de informação necessários: Nome do Componente, Versão, Fonte de Consulta, Identificador do Pacote.

RF_FF04 - Verificar Componentes Desatualizados. Itens de informação necessários: Nome do Componente, Versão Atual Utilizada, Última Versão Disponível, Ecossistema.

RF_FF05 - Gerar Relatório de Análise. Itens de informação necessários: Componentes Identificados, Vulnerabilidades Encontradas, Severidade, Identificadores CVE, Recomendações de Atualização.

1.2.2 Funções Básicas

Refere-se às operações necessárias para configuração, manutenção e execução da ferramenta. São elas:

RF_FB01 - Configurar Fontes de Consulta. Itens de informação necessários: Nome da Fonte, URL da API, Chave de Acesso, Status da Fonte.

RF_FB02 - Manter Banco Local de Vulnerabilidades. Itens de informação necessários: Identificador da Vulnerabilidade, Componente Afetado, Versão Afetada, Severidade, Data de Sincronização.

RF_FB03 - Sincronizar Base Local de Vulnerabilidades. Itens de informação necessários: Fonte de Dados, Data da Última Atualização, Quantidade de Registros, Status da Sincronização.

RF_FB04 - Configurar Modo de Execução. Itens de informação necessários: Caminho do Projeto, Modo Online, Modo Offline, Uso de Cache, Formato do Relatório.

RF_FB05 - Manter Cache de Consultas. Itens de informação necessários: Componente Consultado, Versão, Resultado da Consulta, Data de Armazenamento.

RF_FB06 - Selecionar Ecossistema de Análise. Itens de informação necessários: Tipo de Projeto, Arquivo de Dependência, Parser Utilizado.

RF_FB07 - Validar Arquivos de Dependência. Itens de informação necessários: Caminho do Arquivo, Tipo do Arquivo, Formato Esperado, Status da Validação.

1.2.3 Funções de Saída

RF_FS01 - Exibir Componentes Identificados. Itens de informação exibidos: Nome do Componente, Versão, Ecossistema, Arquivo de Origem.

RF_FS02 - Exibir Vulnerabilidades Encontradas. Itens de informação exibidos: Nome do Componente, Versão Vulnerável, Identificador CVE, Descrição da Vulnerabilidade, Severidade, Fonte da Informação.

RF_FS03 - Exibir Recomendações de Atualização. Itens de informação exibidos: Componente, Versão Atual, Versão Recomendada, Ação Sugerida.

RF_FS04 - Exibir Resumo da Varredura. Itens de informação exibidos: Total de Componentes Analisados, Total de Vulnerabilidades Encontradas, Quantidade por Severidade, Tempo de Execução.

RF_FS05 - Emitir Relatório HTML. Itens de informação exibidos: Dados Gerais do Projeto, Lista de Componentes, Vulnerabilidades, Severidade, Recomendações e Resultado Final da Análise.

1.3 Regras do Negócio

Regras do Negócio

A varredura só será realizada quando o projeto possuir arquivos de dependência compatíveis com os ecossistemas suportados pela ferramenta.

Um componente será considerado vulnerável quando seu nome e versão estiverem associados a uma vulnerabilidade conhecida em bases públicas, como OSV, NVD ou Sonatype Guide.

Um componente será considerado desatualizado quando a versão utilizada no projeto for inferior à versão mais recente disponível.

As vulnerabilidades identificadas deverão ser apresentadas com informações como CVE, severidade, fonte da consulta e recomendação de atualização ou mitigação.

O sistema não realizará correções automáticas nas dependências, sendo responsabilidade do desenvolvedor aplicar as atualizações sugeridas.

O modo offline dependerá da existência de uma base local previamente sincronizada.

1.4 Perspectiva do produto / Requisitos não funcionais

O BasiliskScan será uma ferramenta de linha de comando desenvolvida em Python, voltada à análise de componentes vulneráveis e desatualizados em sistemas web. O sistema será executado diretamente pelo terminal, permitindo que o usuário informe o caminho do projeto a ser analisado e receba como resultado uma varredura das dependências utilizadas. A ferramenta será compatível com os principais sistemas operacionais utilizados em ambientes de desenvolvimento, como Windows e Linux, podendo também ser adaptada para macOS, desde que o ambiente possua o Python e as dependências necessárias instaladas.

O sistema não dependerá de uma interface gráfica tradicional, pois sua interação ocorrerá por meio de comandos no terminal. Mesmo assim, deverá apresentar mensagens claras, organizadas e compreensíveis, indicando o andamento da análise, possíveis erros encontrados e o resultado final da varredura. Além da saída no terminal, a ferramenta deverá gerar relatórios em formato HTML,

permitindo que os dados obtidos sejam visualizados de forma mais acessível e compartilhável.

O BasiliskScan será estruturado de forma modular, com separação entre os módulos responsáveis pela leitura dos arquivos de dependência, consulta às bases de vulnerabilidades, normalização dos resultados, armazenamento local e geração de relatórios. Essa organização permitirá a manutenção do sistema e a inclusão futura de novos ecossistemas de desenvolvimento, sem a necessidade de reescrever toda a aplicação.

A ferramenta deverá oferecer suporte inicial aos ecossistemas Node.js, Ionic/TypeScript e Java Web, analisando arquivos como package.json, pom.xml e build.gradle. A execução da análise deverá ser eficiente, consumindo poucos recursos computacionais, de modo que possa ser utilizada tanto em computadores de maior desempenho quanto em máquinas mais simples. Recomenda-se, para bom funcionamento, um ambiente com pelo menos 4GB de memória principal, espaço disponível para instalação da ferramenta e armazenamento dos relatórios, além de conexão com a internet para consultas às bases públicas de vulnerabilidades.

O sistema poderá utilizar cache e banco de dados local para armazenar informações de vulnerabilidades já consultadas, reduzindo a necessidade de chamadas repetidas às APIs externas e melhorando o desempenho em análises posteriores. Também poderá ser utilizado em modo offline, desde que a base local de vulnerabilidades tenha sido previamente sincronizada. Dessa forma, a ferramenta busca oferecer maior flexibilidade para diferentes ambientes de uso.

Quanto à segurança, o BasiliskScan não deverá modificar automaticamente os arquivos do projeto analisado, nem realizar exploração ativa de vulnerabilidades. Sua função será exclusivamente identificar, classificar e apresentar informações sobre componentes vulneráveis ou desatualizados. As recomendações de atualização ou mitigação deverão ser exibidas ao usuário, cabendo ao desenvolvedor responsável decidir e aplicar as correções necessárias.

A confiabilidade da ferramenta dependerá da correta leitura dos arquivos de dependência e da disponibilidade das bases públicas utilizadas, como OSV, NVD e Sonatype Guide. Caso uma das fontes externas esteja indisponível, o sistema deverá continuar a execução utilizando as demais fontes disponíveis ou os dados previamente armazenados em cache, quando existirem.

A implantação do BasiliskScan contribuirá para o aumento da segurança dos projetos analisados, principalmente no contexto da Fábrica de Software Acadêmica, permitindo maior controle sobre bibliotecas de terceiros, versões utilizadas e vulnerabilidades conhecidas. Por ser uma ferramenta de código aberto, modular e executada por linha de comando, poderá ser utilizada localmente por desenvolvedores, integrada a rotinas de análise e futuramente expandida para outros ecossistemas de software.

1.5 Restrições, Suposições e Dependências

O sistema será desenvolvido na linguagem de programação Python;

A ferramenta será executada por meio de interface de linha de comando, não possuindo interface gráfica tradicional;

O funcionamento do sistema dependerá da existência de arquivos de dependência compatíveis no projeto analisado, como package.json, pom.xml ou build.gradle;

A análise inicial será limitada aos ecossistemas Node.js, Ionic/TypeScript e Java Web;

A consulta de vulnerabilidades dependerá da disponibilidade das bases públicas utilizadas, como OSV, NVD e Sonatype Guide;

Para realizar consultas online, será necessário que o computador esteja conectado à internet;

O modo offline dependerá de uma base local de vulnerabilidades previamente sincronizada;

É de responsabilidade do usuário manter a base local atualizada para que os resultados em modo offline sejam mais confiáveis;

O sistema não realizará correções automáticas nas dependências vulneráveis ou desatualizadas, sendo responsabilidade do desenvolvedor aplicar as atualizações recomendadas;

A ferramenta não deverá modificar os arquivos do projeto analisado, atuando apenas na leitura, identificação e geração de relatórios;

Os resultados apresentados dependerão da qualidade e atualização das informações fornecidas pelas bases de vulnerabilidades consultadas;

A correta identificação dos componentes dependerá da estrutura adequada dos arquivos de dependência do projeto;

O sistema pressupõe que o usuário tenha conhecimento básico sobre terminal, dependências de software e atualização de bibliotecas;

O relatório gerado deverá ser utilizado como apoio à tomada de decisão, não substituindo testes de compatibilidade, revisão de código ou validações de segurança adicionais.

Capítulo 2 - REQUISITOS ESPECÍFICOS

2.1 Diagrama de Casos de Uso

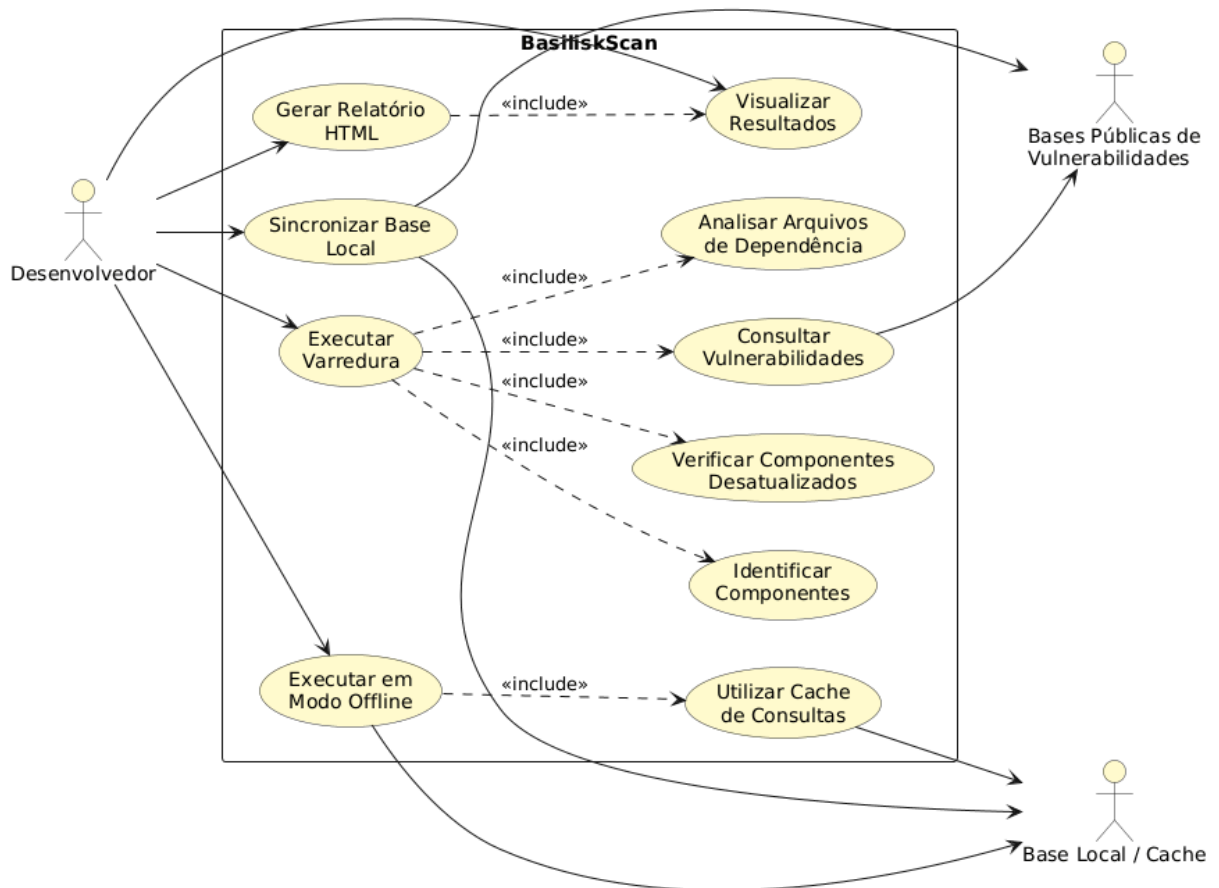


Figura 1: Diagrama de Casos de Uso.

2.2 Especificações de Casos de Uso e Diagramas de Atividades

Especificação do Caso de Uso: Executar Varredura
Referências Cruzadas: RF_FF01, RF_FF02, RF_FF03, RF_FF04

Ator Principal: Desenvolvedor
Interessados e Interesses: - Desenvolvedor: deseja analisar um projeto e identificar componentes vulneráveis ou desatualizados. - Equipe de desenvolvimento: deseja obter informações para corrigir riscos relacionados às dependências utilizadas no projeto.
Pré-condições: O desenvolvedor deve possuir o BasiliskScan instalado e informar um caminho válido para o projeto a ser analisado.
Garantia de sucesso (pós-condições): Os arquivos de dependência do projeto são analisados, os componentes são identificados e as vulnerabilidades ou versões desatualizadas são apresentadas ao usuário.
Fluxo Básico: 1. O caso de uso inicia quando o desenvolvedor executa o comando de varredura no terminal. 2. O sistema recebe o caminho do projeto informado pelo usuário. 3. O sistema verifica se o caminho informado é válido. 4. O sistema identifica o ecossistema do projeto, como Node.js, Ionic/TypeScript ou Java Web. 5. O sistema localiza os arquivos de dependência compatíveis, como package.json, pom.xml ou build.gradle. 6. O sistema realiza a leitura dos arquivos encontrados. 7. O sistema extrai os nomes e versões dos componentes utilizados no projeto. 8. O sistema consulta as bases de vulnerabilidades configuradas. 9. O sistema verifica se os componentes identificados possuem vulnerabilidades conhecidas ou versões desatualizadas. 10. O sistema normaliza os resultados encontrados. 11. O sistema exibe um resumo da varredura no terminal.
Fluxos Alternativos: Caminho inválido: caso o caminho informado não exista ou não possa ser acessado, o sistema exibe uma mensagem de erro e encerra a operação. Arquivo de dependência não encontrado: caso nenhum arquivo compatível seja localizado, o sistema informa que o projeto não pôde ser analisado. Falha em base externa: caso uma base de vulnerabilidades esteja indisponível, o sistema continua a análise com as demais fontes disponíveis ou com dados em cache, quando existirem.

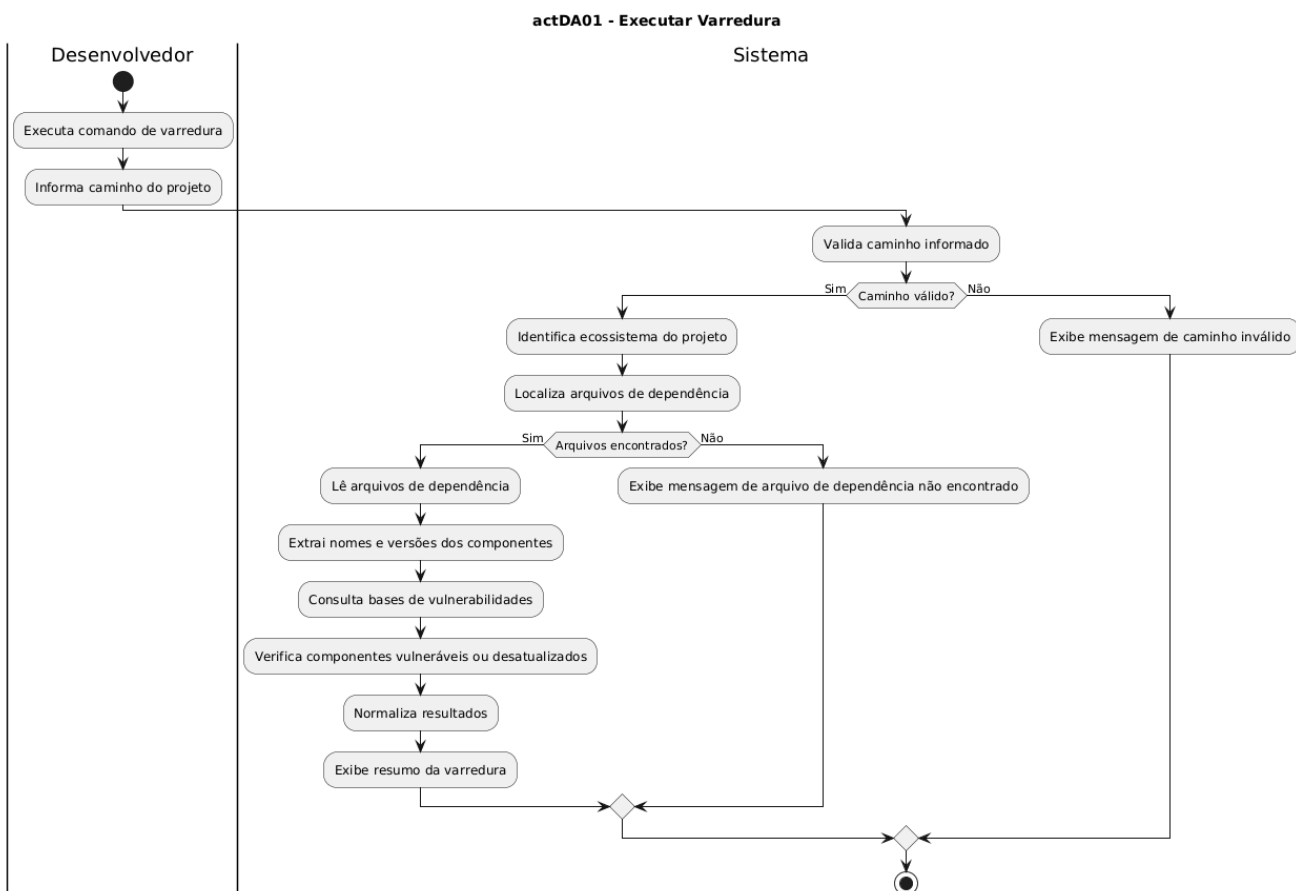


Figura 2: Diagrama de Atividade - Executar Varredura.

Especificação do Caso de Uso: Gerar Relatório HTML
Referências Cruzadas: RF_FF05, RF_FS01, RF_FS02, RF_FS03, RF_FS04, RF_FS05
Ator Principal: Professor
Interessados e Interesses: - Desenvolvedor: deseja obter um relatório visual e organizado sobre os resultados da análise. - Equipe de desenvolvimento: deseja consultar componentes, vulnerabilidades e recomendações de atualização de forma clara.
Pré-condições: - Uma varredura deve ter sido executada e os resultados da análise devem estar disponíveis.
Garantia de sucesso (pós-condições):

Um relatório HTML é gerado contendo informações sobre o projeto analisado, componentes identificados, vulnerabilidades encontradas, severidades e recomendações.

Fluxo Básico:

1. O caso de uso inicia quando o desenvolvedor solicita a geração do relatório HTML.
2. O sistema recupera os dados obtidos durante a varredura.
3. O sistema organiza os componentes identificados por nome, versão e ecossistema.
4. O sistema organiza as vulnerabilidades encontradas, incluindo CVE, descrição, severidade e fonte da informação.
5. O sistema adiciona recomendações de atualização ou mitigação quando disponíveis.
6. O sistema calcula o resumo geral da análise, como total de componentes e total de vulnerabilidades.
7. O sistema gera o arquivo HTML do relatório.
8. O sistema informa ao usuário o local em que o relatório foi salvo.

Fluxos Alternativos:

Nenhum resultado disponível: caso não exista uma varredura anterior, o sistema informa que é necessário executar uma análise antes de gerar o relatório.

Falha ao salvar o arquivo: caso o sistema não consiga gravar o relatório no diretório de destino, será exibida uma mensagem de erro.

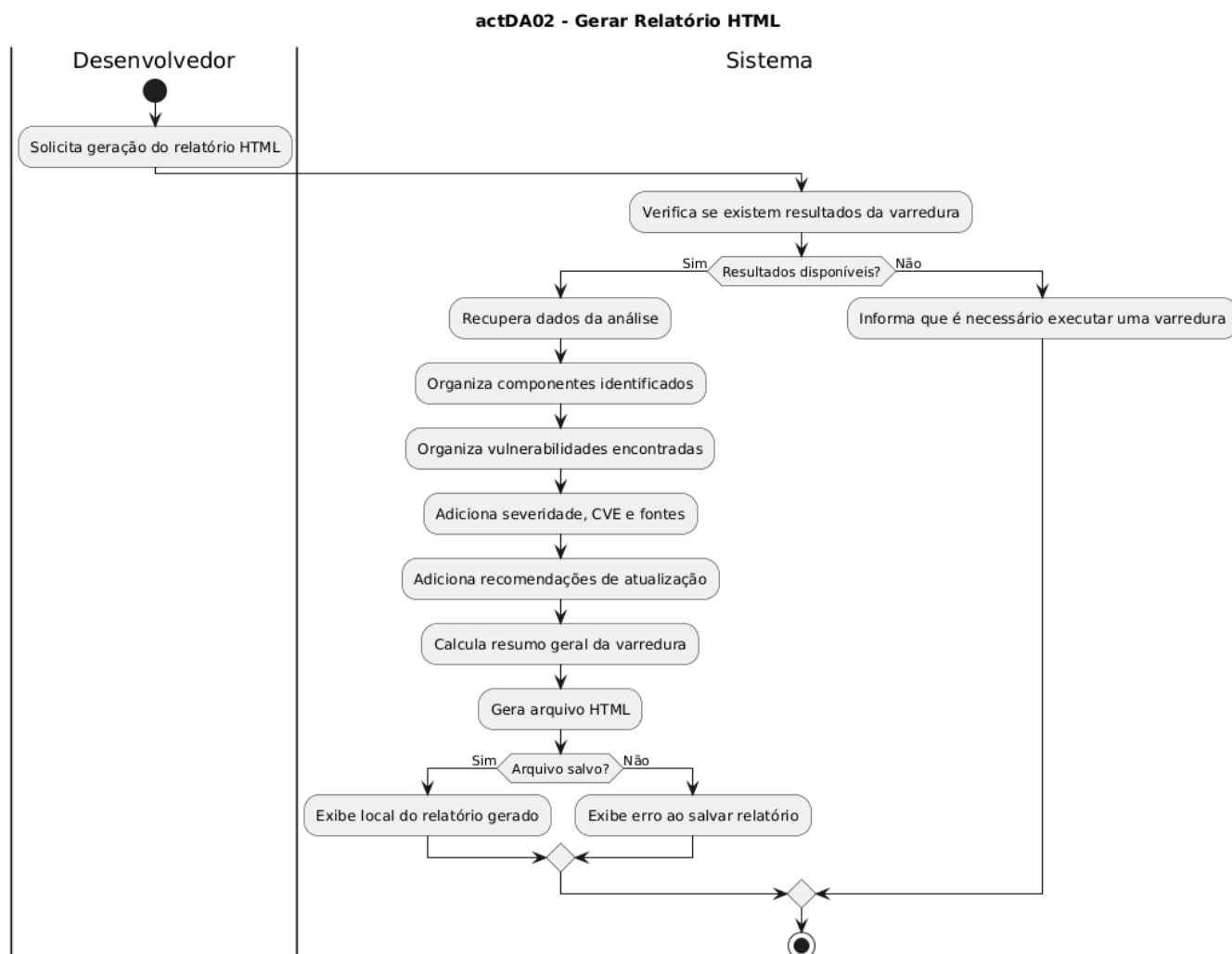


Figura 3: Diagrama de Atividade - Gerar Relatório HTML .

Especificação do Caso de Uso: Sincronizar Base Local

Referências Cruzadas: RF_FB02, RF_FB03

Ator Principal: Desenvolvedor

Interessados e Interesses:

- Desenvolvedor: deseja manter uma base local de vulnerabilidades atualizada para melhorar o desempenho e permitir análises futuras.
- Sistema: necessita armazenar informações locais para reduzir chamadas repetitivas às bases externas.

Pré-condições:

O sistema deve estar instalado corretamente e o computador deve possuir conexão com a internet.

Garantia de sucesso (pós-condições):

A base local de vulnerabilidades é atualizada com os dados obtidos das fontes configuradas.

Fluxo Básico:

1. O caso de uso inicia quando o desenvolvedor solicita a sincronização da base local.
2. O sistema verifica a conexão com a internet.
3. O sistema acessa as fontes públicas de vulnerabilidades configuradas.
4. O sistema baixa ou atualiza os registros disponíveis.
5. O sistema armazena os dados na base local.
6. O sistema registra a data e o status da sincronização.
7. O sistema informa ao desenvolvedor que a sincronização foi concluída.

Fluxos Alternativos:

Sem conexão com a internet: o sistema informa que a sincronização não pode ser realizada no momento.

Falha em uma fonte externa: o sistema registra a falha e continua a sincronização com as demais fontes disponíveis.

Base local indisponível: caso não seja possível acessar ou gravar na base local, o sistema exibe uma mensagem de erro.

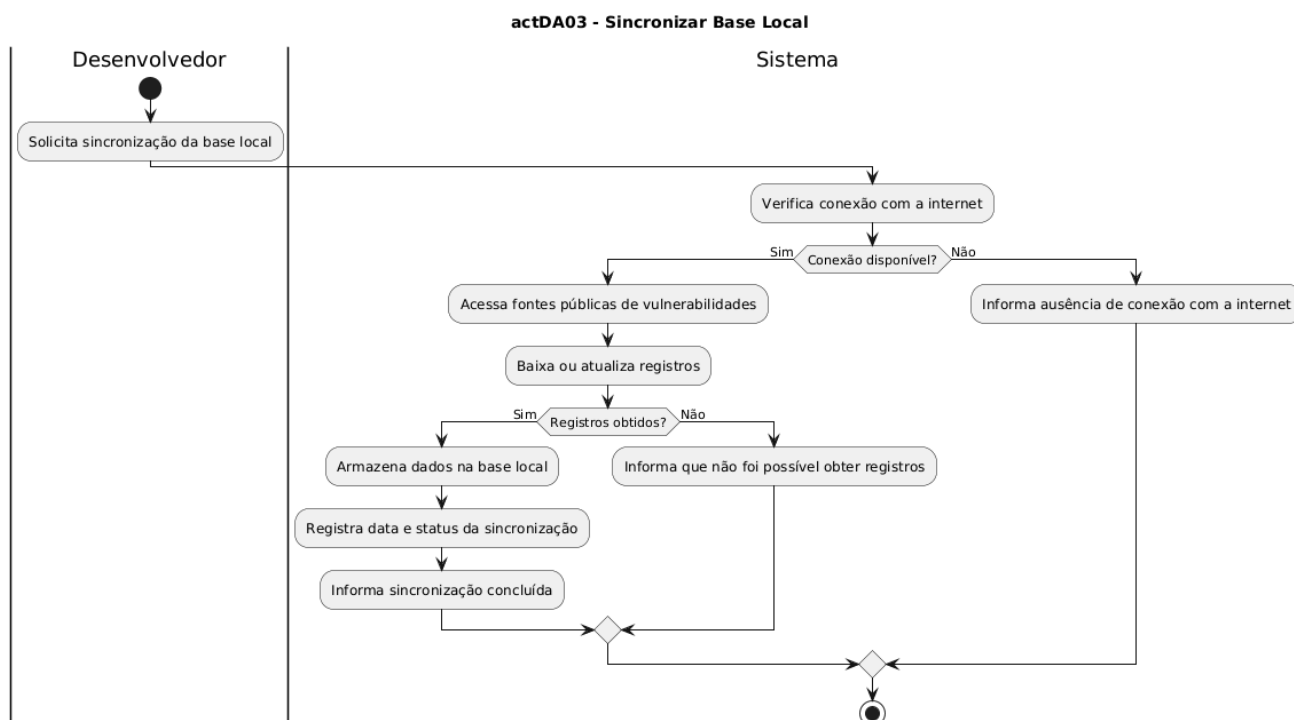


Figura 4: Diagrama de Atividade - Sincronizar Base Local.

Especificação do Caso de Uso: Executar em Modo Offline.

Referências Cruzadas: RF_FB02, RF_FB04, RF_FB05

Ator Principal: Desenvolvedor

Interessados e Interesses:

- Desenvolvedor: deseja realizar a análise do projeto sem depender de conexão com a internet.
- Equipe de desenvolvimento: deseja obter resultados mesmo em ambientes restritos ou com conexão instável.

Pré-condições:

- A base local de vulnerabilidades deve ter sido previamente sincronizada.

Garantia de sucesso (pós-condições):

O projeto é analisado utilizando apenas os dados armazenados localmente, sem consultas às bases externas.

Fluxo Básico:

1. O caso de uso inicia quando o desenvolvedor executa a varredura em modo offline.
2. O sistema verifica se existe uma base local de vulnerabilidades disponível.
3. O sistema recebe o caminho do projeto informado.
4. O sistema identifica os arquivos de dependência compatíveis.
5. O sistema extrai os componentes e versões utilizados.
6. O sistema consulta a base local de vulnerabilidades.
7. O sistema identifica componentes vulneráveis ou desatualizados conforme os dados locais disponíveis.
8. O sistema exibe os resultados da análise.
9. O sistema pode gerar um relatório com base nos dados encontrados.

Fluxos Alternativos:

Base local inexistente: caso a base local não exista, o sistema informa que é necessário realizar uma sincronização antes de utilizar o modo offline.

Base local desatualizada: o sistema poderá alertar que os resultados podem não refletir as vulnerabilidades mais recentes.

Projeto inválido: caso o caminho do projeto seja inválido ou não contenha arquivos compatíveis, o sistema informa o erro ao usuário.

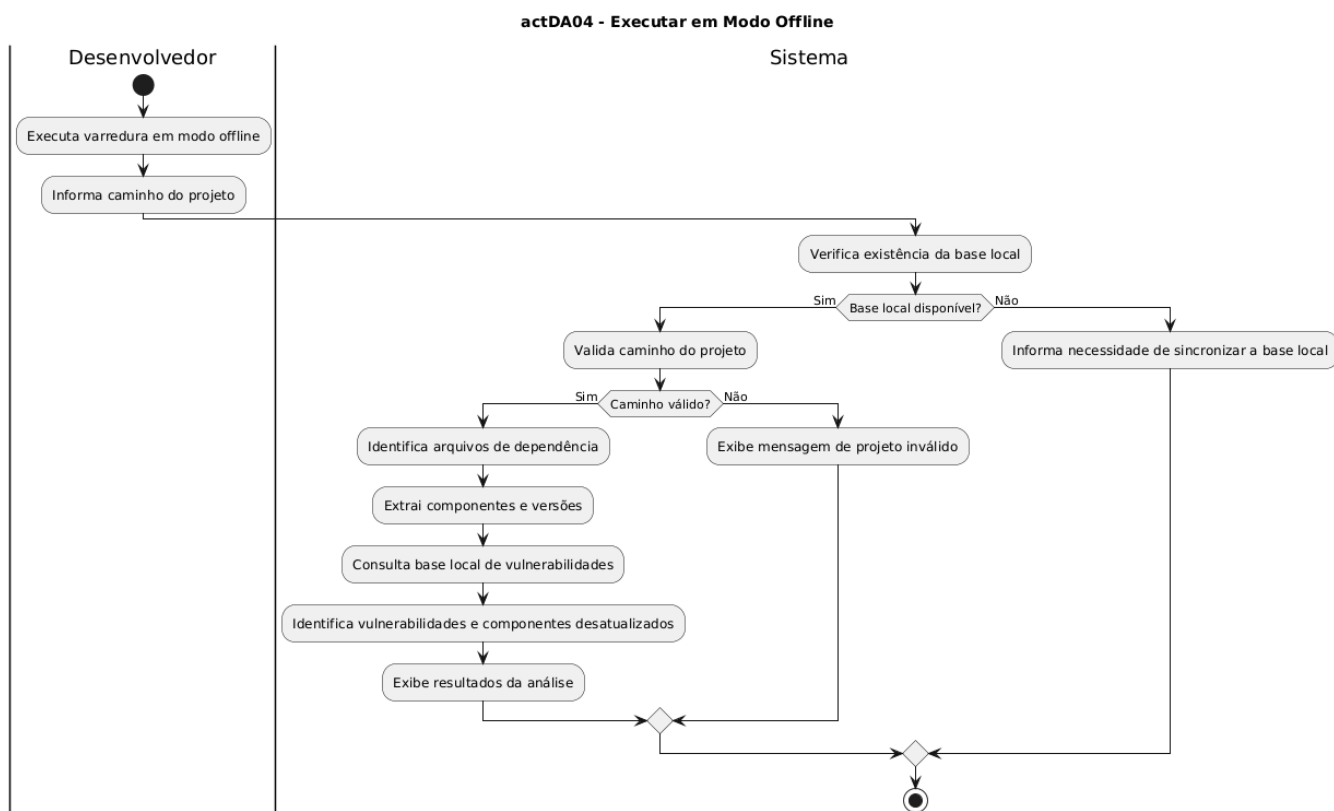


Figura 5: Diagrama de Atividade - Executar em Modo Offline.

Capítulo 3 - PROJETO DE SOFTWARE

3.1 Arquitetura Lógica de Software

Arquitetura Lógica de Software - BasiliskScan

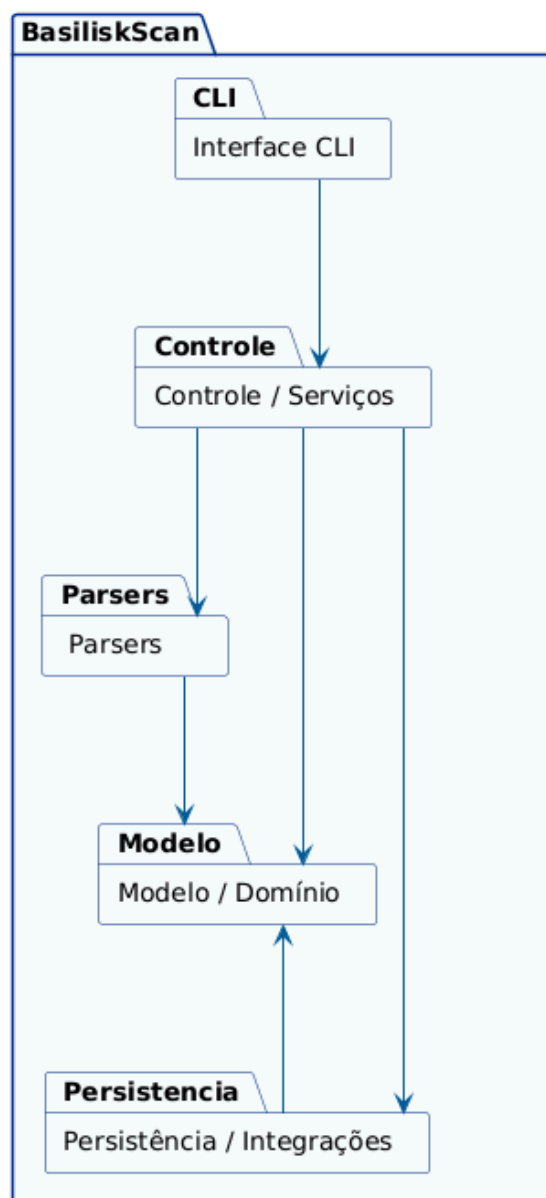


Figura 6: Arquitetura Lógica de Software.

As camadas definidas na arquitetura lógica do BasiliskScan são Interface CLI, Controle, Parsers, Modelo e Persistência. A camada de Interface CLI é responsável pela interação direta com o usuário, concentrando os comandos utilizados para executar varreduras, gerar relatórios e acionar funcionalidades offline da ferramenta.

A camada de Controle possui os serviços responsáveis por coordenar o

funcionamento da aplicação. Nela ficam concentradas as regras de execução das operações, como o acionamento da varredura, a chamada dos parsers, a normalização dos resultados, a geração de relatórios e a comunicação com os mecanismos de persistência e integração.

A camada de Parsers é responsável pela leitura e interpretação dos arquivos dos projetos analisados. Essa camada identifica os arquivos de dependências de cada ecossistema suportado, como Node.js, Ionic/TypeScript e Java, extraindo as informações necessárias para que os componentes utilizados pelo sistema sejam representados internamente.

A camada de Modelo representa as entidades principais do domínio da aplicação, como projeto, componente, dependência, vulnerabilidade e resultado da análise. Essas entidades organizam os dados obtidos durante a varredura e servem como base para o processamento das informações dentro da ferramenta.

A camada de Persistência concentra os recursos responsáveis pelo armazenamento local, cache e integração com bases externas de vulnerabilidades. Nela estão os mecanismos utilizados para consultar, armazenar e recuperar informações provenientes de fontes como OSV, NVD, Sonatype Guide e banco offline.

A comunicação entre as camadas ocorre a partir da Interface CLI, que recebe os comandos do usuário e os encaminha para a camada de Controle. O Controle coordena a execução da análise, aciona os Parsers para realizar a leitura dos arquivos do projeto e manipula os dados representados na camada de Modelo. A camada de Modelo se relaciona com a Persistência para armazenar e recuperar informações sobre componentes e vulnerabilidades. Dessa forma, a arquitetura separa as responsabilidades da ferramenta, facilitando a manutenção, a organização do código e a inclusão futura de novos parsers, serviços e fontes de dados.

3.2 Diagrama de Classes

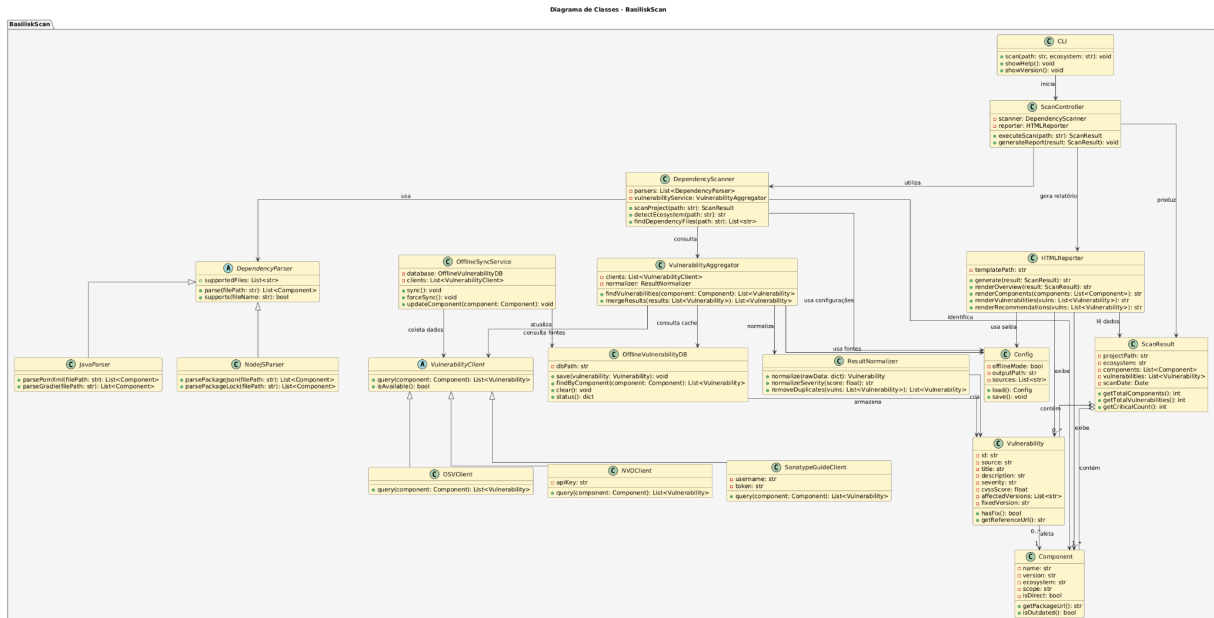


Figura 7: Diagrama de Classes - Modelo.

3.3 Mapeamento OO-Relacional (Modelo Lógico de Dados)

SCAN = (PK_idScan, caminhoProjeto, nomeProjeto, ecossistema, dataScan, statusScan, tempoExecucao, totalComponentes, totalVulnerabilidades)

ARQUIVODEPENDENCIA = (PK_idArquivoDependencia, nomeArquivo, caminhoArquivo, tipoArquivo, FK_scan_idScan)

COMPONENTE = (PK_idComponente, nome, versao, ecossistema, escopo, tipoDependencia, packageUrl, desatualizado, versaoMaisRecente)

```
COMPONENTESCAN      =      (PK_idComponenteScan,      FK_scan_idScan,
FK_componente_idComponente,
FK_arquivoDependencia idArquivoDependencia)
```

VULNERABILIDADE = (PK_idVulnerabilidade, identificador, titulo, descricao, severidade, cvssScore, versaoCorrigida, urlReferencia)

COMPONENTEVULNERABILIDADE = (PK idComponenteVulnerabilidade,

FK_componente_idComponente, FK_vulnerabilidade_idVulnerabilidade)

FONTEVULNERABILIDADE = (PK_idFonte, nomeFonte, urlBase, tipoFonte, status)

CONSULTAVULNERABILIDADE = (PK_idConsulta, dataConsulta, statusConsulta, respostaNormalizada, FK_componente_idComponente, FK_fonte_idFonte)

RELATORIO = (PK_idRelatorio, caminhoArquivo, formato, dataGeracao, FK_scan_idScan)

RECOMENDACAO = (PK_idRecomendacao, descricao, tipoRecomendacao, prioridade, FK_vulnerabilidade_idVulnerabilidade, FK_componente_idComponente)

BANCOOFFLINE = (PK_idBancoOffline, caminhoBanco, dataUltimaSincronizacao, totalRegistros, status)

SINCRONIZACAO = (PK_idSincronizacao, dataInicio, dataFim, statusSincronizacao, totalItensAtualizados, FK_bancoOffline_idBancoOffline)